

WP1 — Bảng tra cứu 12 file

WP1 — Bảng tra cứu 12 file dụng môi trường

Đồ án NT219: Benchmark ML-KEM/ML-DSA (Kyber/Dilithium) vs RSA/ECC. File này dành cho thành viên nhóm: mỗi file làm gì, chạy thế nào, kiểm ra sao, khác gì giữa x86_64 và ARM. Mọi thứ cài vào ~/pqc (KHÔNG đụng OpenSSL hệ thống). Script duy nhất cần sudo là `install_prereqs.sh`.

0. Thứ tự chạy chuẩn

Trên PC x86_64:

```
cd <gốc repo>
chmod +x scripts/*.sh
bash scripts/install_prereqs.sh
bash scripts/build_openssl.sh
source scripts/setenv.sh
bash scripts/verify_env.sh
make
bash scripts/build_liboqs.sh ref
bash scripts/fetch_pqclean.sh clean
```

*# một lần (file đi từ Windows sang máy Linux)
1) toolchain + công cụ đo (sudo)
2) OpenSSL 3.6.2 PQC native (~15-30 phút)
3) bật môi trường (MỖI cửa sổ term)
4) cổng gác + biên bản môi trường
5) build bench_evp (WP2)
6) liboqs cây ref (nghiệm thu)
7) PQClean bản clean (WP5 code-size)*

Trên máy ARM thuê (Oracle Ampere/AWS t4g — chọn image Ubuntu):

```
git clone <repo> && cd <repo> && chmod +x scripts/*.sh
bash scripts/install_prereqs.sh
bash scripts/build_openssl.sh
source scripts/setenv.sh
bash scripts/verify_env.sh
bash scripts/build_liboqs.sh ref
bash scripts/build_liboqs.sh opt
bash scripts/fetch_pqclean.sh clean
bash scripts/fetch_pqclean.sh aarch64
```

*# build LẠI từ đầu (binary không mang theo)
-> docs/env_report_aarch64.txt
BẮT BUỘC cả hai cây = thí nghiệm NEON
trực đối chứng NEON thứ hai*

1. Bảng 12 file

#	File — vị trí	Làm gì trong đồ án	Lệnh chạy + tham số	Sinh ra ở đâu	Cách kiểm tra
1	versions.env — scripts/	Nguồn-sự-thật- duy-nhất: pin tag (OpenSSL 3.6.2, liboqs 0.14.0, oqs-provider 0.10.0) + mọi đường dẫn ~/pqc (Re- producibility, đề bài §7.3)	Không chạy trực tiếp — mọi script tự source. Không tham số	Không sinh gì (file đầu vào)	bash -n scripts/versions. dò biến chết: grep -rl '\\${TÊN_BIẾN}' -- include='*.sh' . phải ≥ 1 file
2	setenv.sh — scripts/	Bật OpenSSL custom cho terminal hiện tại (PATH + LD_LIBRARY_PATH không rpath — theo quy ước demo OpenSSL)	source scripts/setenv.sh — PHẢI source, không execute, mỗi cửa sổ terminal. Không tham số	Chỉ sinh biến môi trường trong shell	which openssl → ~/pqc/openssl/bin/ openssl version → 3.6.2
3	install_prerequisites — scripts/	Cài toolchain (gcc/cmake/ninja/ perl/git/ §8.2) + công cụ đo (time, htop, ab, wrk, tshark, perf) + pan- das/matplotlib (§16). Script duy nhất cần sudo	bash scripts/install_prerequisites.sh (hỏi mật khẩu sudo; hộp thoại tshark → chọn Yes). Không tham số	Gói hệ thống vào /usr reqs.sh	gcc -shversion && cmake --version && /usr/bin/time --version
4	build_openssl — scripts/	Build OpenSSL 3.6.2 PQC native (ML- KEM/ML-DSA có sẵn trong libcrypto) — nền của WP2/WP4/WP5	bash scripts/build_openssl.sh Env tùy chọn: SKIP_TESTS=1 (bỏ test suite, nhANH), FORCE=1 (ép build lại)	Cài openssl nguồn bằng chứng docs/openssl	Script tự verify rồi in DONE; kiểm trá; ~/pqc/openssl/bin/ ls -l item- algorithms \ grep ML-KEM

#	File — vị trí	Làm gì trong đồ án	Lệnh chạy + tham số	Sinh ra ở đâu	Cách kiểm tra
5	build_liboqs- scripts/	Build liboqs hai cây ref/opt với OQS_DIST_BUILD — trái tim WP3 (RQ2: hiệu ứng NEON)	bash scripts/build- ref\ opt — tham số bắt buộc . Env: FORCE=1; USE_ARM_PMU=1 (chỉ ARM, cần nạp kernel module pqax trước, VPS thuê thường không được)	~/pqc/liboqs- ref\ opt- ~/pqc/liboqs*/lib/liboqs.so; neon;/ speed tools ở ~/pqc/src/liboqs- variant>/tests/ docs/liboqs- header	chạy speed_kem liboqs-build- tests/đọc header OpenSSL enabled: 3.6.2, CPU exts đúng cây (ref: SSE SSE2; opt: AVX2... hoặc NEON)
6	fetch_pqclean- scripts/	Clone PQClean + build thư viện .a tách riêng từng thuật toán — checkbox §7.2, code-size per-algorithm (WP5), trực đối chứng NEON thứ hai (WP3). Lưu ý: PQClean sẽ archive read-only 7/2026, kế nhiệm PQ Code Package — ghi vào Literature	bash scripts/fetch- pqclean.sh [clean\ avx2clean\ no- — mặc định clean; có guard chặn variant sai má	~/pqc/src/PQClean- clean\ avx2clean\ no- docs/pqclean- variant>/tests/ ~/pqc/src/PQClean- kem- 768/clean/lib*.a	Thấy trong crypto_*/ml lib*.a; commit ~/pqc/src/PQClean- kem- 768/clean/lib*.a
7	build_oqsprovider- scripts/	Build OQS Provider — kế nhiệm chính thức của fork OQS-OpenSSL mà đề bài nêu (§7.2); nhánh thí nghiệm TLS-qua- liboqs	bash scripts/build- oqsprovider- [ref\ opt] — mặc định opt. Điều kiện: OpenSSL + liboqs đúng variant đã build (2 cổng gác tự chặn kèm chỉ dẫn)	~/pqc/oqs- provider- liboqsprovider- docs/oqsprovider- variant>/tests/ ~/pqc/oqs- provider- liboqsprovider- variant>/tests/ ~/pqc/oqs- provider- liboqsprovider- variant>/tests/	Script tự lấy openssl liboqsprovider.so; load, kiểm tra openssl list -providers -provider- path ~/pqc/oqs- provider/lib -provider oqsprovider

#	File — vị trí	Làm gì trong đồ án	Lệnh chạy + tham số	Sinh ra ở đâu	Cách kiểm tra
8	cross_build_— scripts/ Cross compile liboqs aarch64 ngay trên PC — bằng chúng đáp ứng về “separate cross-compile scripts (aarch64-linux- gnu toolchain)” của §7.3. KHÔNG dùng binary này để đo		bash scripts/cross_build_— liboqs Không tham số. Điều kiện: sudo apt install gcc- aarch64- linux-gnu g++- aarch64- linux-gnu (guard tự chặn nếu thiếu)	~/pqc/liboqs aarch64-liboqs. cross/ (binary ARM generic)	Đòng cuối files. .../li- boqs.so in “ARM aarch64”
9	aarch64- toolchain.cmake — cmake/	Định nghĩa các toolchain x86→aarch64 theo chuẩn CMake (đồng mẫu toolchain_rasppi.cmake mà liboqs ship)	Không chạy trực tiếp — file #8 nạp qua — DCMAKE_TOOLCHAIN_FILE. Không tham số	Không sinh gì	cmake -P cmake/aarch64- toolchain.cmake im lạng = OK
10	docker_build_— scripts/ Build image đa kiến trúc amd64+arm64 — về “docker buildx multiarch” của §7.3 (tùy chọn, dư điểm vì đã có #8)		bash scripts/docker_build_— Không tham số. Điều kiện: Docker (cài theo docs.docker.com) + QEMU: docker run -- privileged --rm tonisti- igi/binfmt --install all	Image ker219-buildx.sh pqc:multi	docker images \\ grep nt219-pqc

#	File — vị trí	Làm gì trong đồ án	Lệnh chạy + tham số	Sinh ra ở đâu	Cách kiểm tra
11	Dockerfile.x86_64 — docker/	Mô tả công việc đóng hộp — yêu cầu <i>nguyên văn</i> §7.3 “Provide Dockerfile for x86_64”. Tải dùng chính scripts của repo (SKIP_TESTS=1) + cổng verify_env nội bộ	docker build -f docker/Dockerfile.x86_64 -t nt219-pqc . — chạy từ gốc repo . Không tham số	Image nt219-pqc	docker run -it --rm nt219-pqc rồi openssl version → 3.6.2; build tự fail nếu môi trường hỏng
12	verify_env.sh — scripts/	Cổng chống crypto-failure (chặn nếu thiếu ML-KEM/ML-DSA) + biên bản môi trường (gcc, cờ build, CPU governor) — trả lời trực tiếp câu “Document compiler versions, flags, and CPU governor settings” của §7.3	bash scripts/verify_env.sh. Không tham số. Điều kiện: OpenSSL đã build (thiếu thì chính nó báo kèm chỉ dẫn)	docs/env_repo/verify_env.sh	Dùng Pass>.txt Report written to ...; mở file report đối chiếu

2. Ma trận x86_64 vs ARM

File	PC x86_64	Máy ARM thuê (image Ubuntu)
1 versions.env, 2 setenv.sh	Dùng y hệt	Dùng y hệt (theo repo)
3 install_prereqs.sh	Chạy một lần	Chạy một lần — gói linux-tools-\$(uname -r) phát huy trên kernel cloud (-aws/-oracle)
4 build_openssl.sh	Bắt buộc	Bắt buộc — build lại từ đầu , binary không mang chéo máy; chậm hơn PC
5 build_liboqs.sh	ref đủ nghiệm thu; opt tùy chọn (dữ liệu phụ: hiệu ứng AVX2/tối ưu x86)	Bắt buộc CẢ HAI ref + opt — đây chính là thí nghiệm NEON của RQ2
6 fetch_pqclean.sh	clean (+ avx2 nếu muốn so code-size)	clean + aarch64 — guard chặn nếu gõ chéo sân
7 build_oqsprovider.sh	Khi cần provider cho thí nghiệm TLS-liboqs	Tương tự, trên ARM

File	PC x86_64	Máy ARM thuê (image Ubuntu)
8 cross_build_liboqs.sh, 9 toolchain	Chỉ chạy ở đây (cần cross-gcc)	Vô nghĩa — đã trên ARM thì build native
10 docker_buildx.sh, 11 Dockerfile	Nơi có Docker	Không cần — image arm64 sinh từ x86 qua buildx
12 verify_env.sh	→ docs/env_report_x86_64.txt	→ docs/env_report_aarch64.txt — nộp cả hai biên bản

3. Hồ sơ Reproducibility (sinh tự động trong docs/)

Sáu file bằng chứng tự sinh khi chạy, nộp kèm báo cáo: openssl.commit, liboqs.commit, pq-clean.commit, oqsprovider.commit (commit chính xác đã build) + env_report_x86_64.txt, env_report_aarch64.txt (compiler/cờ/governor mỗi máy).

4. Ghi chú nhanh cho người mới vào repo

- **Source vs execute:** setenv.sh phải source (chạy ./setenv.sh là biến mất theo tiến trình con). versions.env và aarch64-toolchain.cmake không bao giờ chạy trực tiếp.
- **Idempotency:** chạy lại script build mà thấy already installed ... Skipping là **đúng thiết kế**, không phải lỗi; muốn build lại thật thì FORCE=1.
- **Đổi phiên bản thư viện:** sửa tag trong scripts/versions.env, xóa thư mục nguồn tương ứng trong ~/pqc/src/, chạy lại với FORCE=1.
- **File .sh từ Windows:** nếu gặp bad interpreter ^M → sed -i 's/\r\$//' scripts/*.sh; thiếu quyền chạy → chmod +x scripts/*.sh.
- **Tuyệt đối không commit** khóa riêng (*.key) — .gitignore đã chặn sẵn.